

CubeAvg

*A project report submitted
in partial fulfillment of the requirement
for the degree in Bachelor's of Computer Application*

By
Aayan Krishna Gogoi
P2200160
22003431



Department of Computer Application,
St. Edmund's College, Shillong

Certificate



This is to certify that the Project work titled '**CubeAvg**' is a bona fide work done by **Aayan Krishna Gogoi, P2200160**, under my guidance during the final semester of the course in partial fulfillment of the Degree of Bachelor of Computer Application.

Ms. Sarmistha Deb
Head of the Department,
BCA

Mr. Melvin Star Majaw
Internal Guide

External Examiner

Date: 13th June 2025

Acknowledgment

I would like to express my sincere gratitude to **Mr. Melvin Star Majaw**, my project guide, for his invaluable guidance, continuous support, and unwavering encouragement throughout the development of this project. His deep insight, patient mentorship, and constructive feedback were instrumental in helping me navigate challenges and achieve successful completion of this work. His belief in my potential has been a constant source of motivation and personal growth.

I am also deeply thankful to our **Head of Department, Ms. Sarmistha Deb**, for providing a supportive academic environment and for her encouragement throughout this project.

My heartfelt thanks to **Br. Sunil Britto**, the **Principal of St. Edmund's College**, for his leadership and for facilitating the infrastructure and academic resources that made this project possible.

To all who guided and supported me on this journey, thank you.

Date: 13th June 2025

Aayan Krishna Gogoi
BCA 6th Sem

Table of Contents

<i>Content</i>	<i>Page No.</i>
Introduction	4
Synopsis	5-10
System Flow Chart	11
Module Design	12
Future Enhancements	13
Conclusion	14
Bibliography	15-16
Source Code	17-30
Sample Input/Output Screens	31-39

Introduction (CubeAvg)

CubeAvg is a mobile application that utilizes image processing and OCR (object/optical character recognition) to extract and analyze data from WCA (World Cube Association) scorecards. The primary objective is to develop an image processing model that accurately recognizes and retrieves handwritten times from scorecards, computes the best and average solve times, and presents the results in a user-friendly interface.



Synopsis

The major objective of this project is to develop a software mobile application that will provide quick results display to all the WCA participants. CubeAvg uses image processing that automates the extraction and analysis of WCA scorecards.

System Study

1. Existing System

I. Manual entry of WCA scores.

- Scores are written by hand on the WCA scorecards
- Then, the scorecards are collected by one official
- Finally, the score of each individual is manually uploaded to the official website from one computer.

Photos:

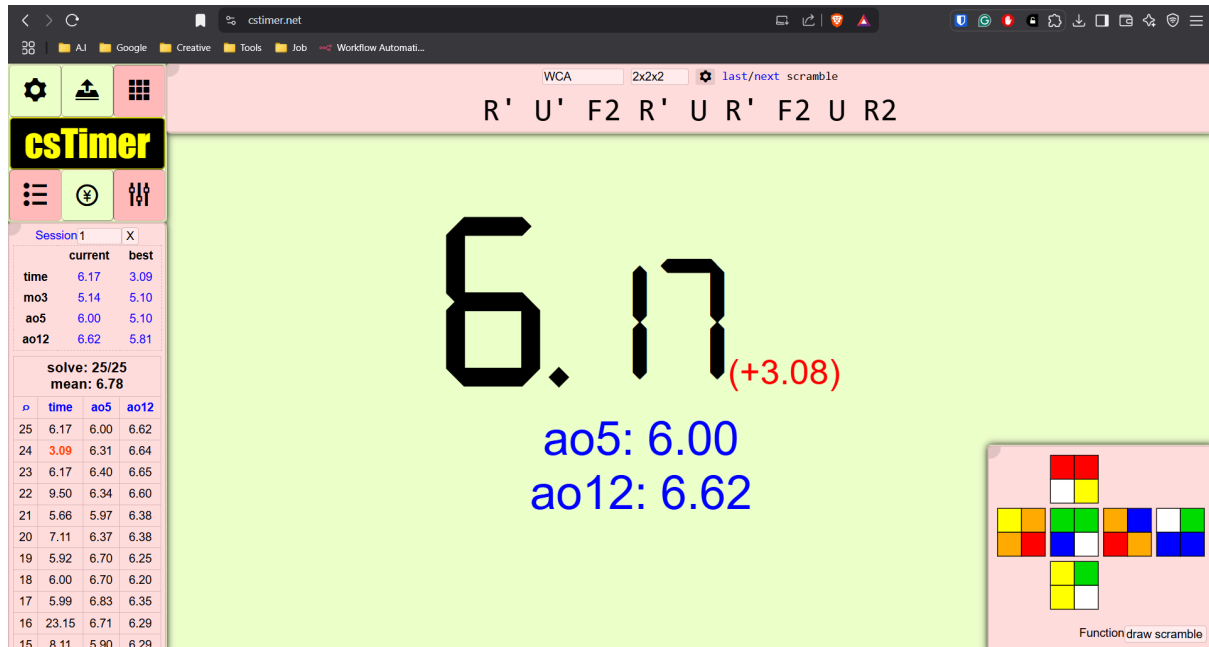
Cubing competition			
Name:		Group	Round
Event:			
		Comp	Judge
1			
2			
3			
4			
5			
6			

Cubing competition			
Name:		Group	Round
Event:			
		Comp	Judge
1	14.05		
2	16.27		
3	11.02		
4	9.98		
5	14.27		
6			

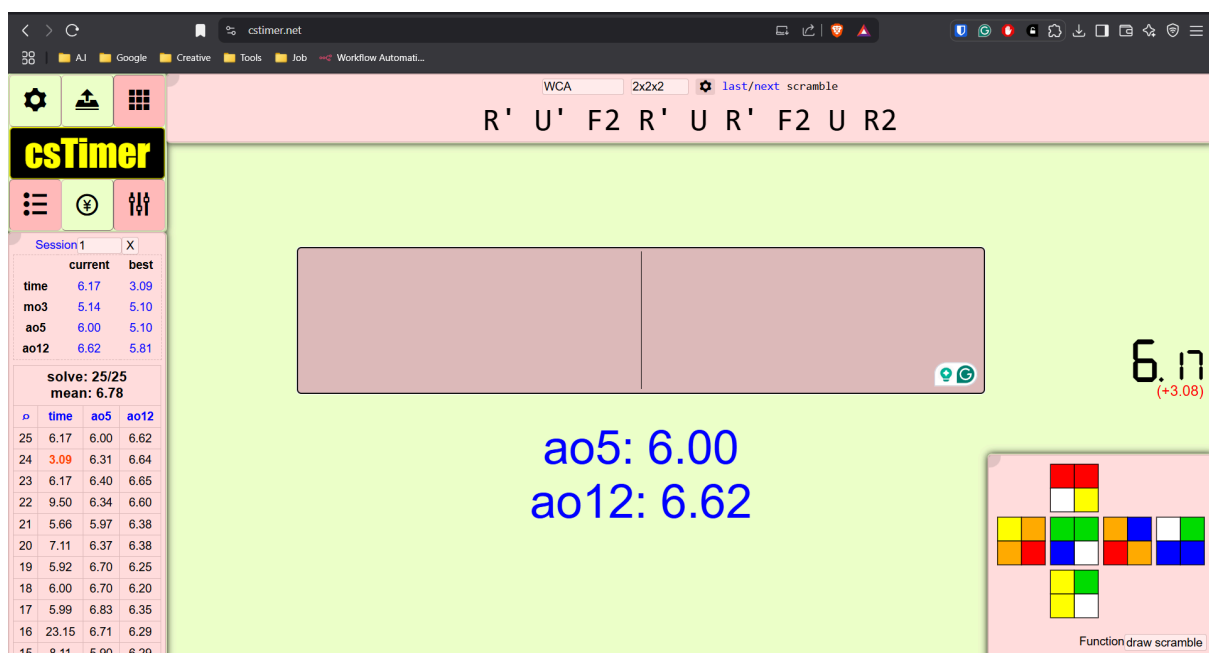
II. *cstimer.net*

- This is a web-based tool that is used for practicing speed cubing solves.
- On this website, there is an option to input data(timings of solves) manually, and users can get their best, average, and worst after 5 data entries.

Photos:



CS Timer Screen



CS Timer User Input

2. Proposed System: CubeAvg

CubeAvg is a mobile application designed to simplify and speed up result processing in WCA competitions. Users can either **upload** or **capture** an image of their WCA scorecard through the app. This image is sent to a backend server where the solve times are **extracted from the scorecard**.

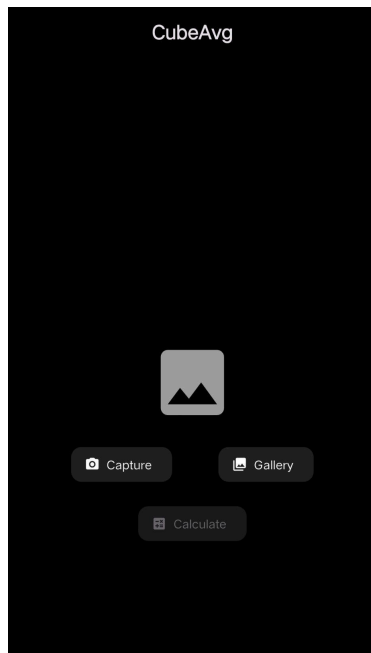
The system then calculates the following: **Best Time** (fastest solve), **Worst Time** (slowest solve), and **Average Time** (based on WCA Ao5 format)

The final results are **instantly displayed** to the user in a card view within the app. This system reduces manual effort and helps make WCA competitions **more efficient and error-free**.

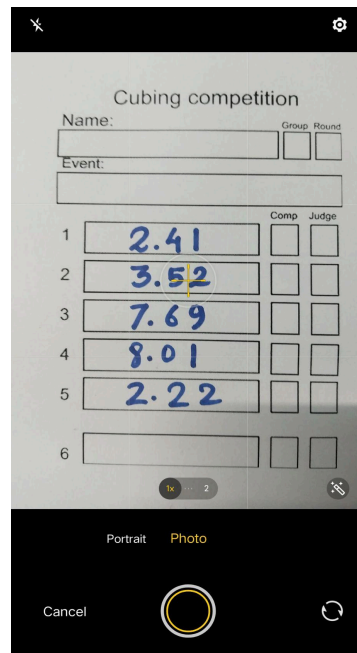
Photos:



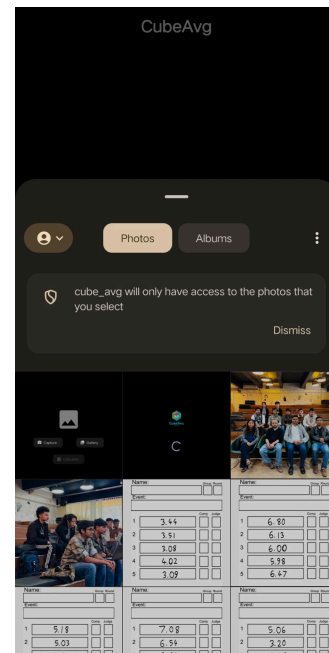
Logo



Home Screen



Capture Screen



Upload from the Gallery Screen

Comparison Between CubeAvg and Existing Solutions

Feature	CubeAvg	Manual Entry	Existing Apps
Auto Time Extraction	✓ Yes	✗ No	✗ No
Error Reduction	✓ Yes (minimizes human error)	✗ No (prone to mistakes)	✗ No (OCR tools are not specialized)
Calculation of Best, Worst, and Average Times	✓ Automatic	✓ Possible (but manual input needed)	✓ Yes (only if manually entered)
Speed & Efficiency	✓ Fast	✗ Slow	✗ Moderate (depends on user)
Designed for WCA Scorecards	✓ Yes	✓ Yes (but manual)	✗ No (generalized timers, OCR)

Feasibility Study

Technical Feasibility

- **Frontend:** Flutter (Dart) – Cross-platform mobile UI development.
- **Backend:** Python with Flask – Lightweight API for handling image processing.
- **OCR:** Tesseract – Accurate and free OCR engine for handwritten text.
- **Image Processing:** OpenCV and Pillow for thresholding and contour detection.

Economic Feasibility

- No additional hardware or software is required beyond a standard Android smartphone and a basic computer for backend hosting.
- All technologies used (Flutter, Python, Flask, Tesseract, OpenCV) are open-source and free to use.
- No licensing or subscription costs are involved in development or deployment.
- The app will be made freely available as an **open-source project**.
- Reduces manual work and time spent in WCA competitions, saving organizational effort and cost.
- Cost-effective solution for both individual cubers and competition organizers.

Operational Feasibility

- The application is simple and user-friendly, with an intuitive interface built using Flutter.
- Users can easily capture or upload images and view results without needing technical knowledge.
- No special training or new skills are required to use the system.
- The process is fully guided within the app, ensuring smooth usage for first-time users.
- Specifically designed for **WCA competitors**, but can also be used by casual cubers and organizers.
- Can be integrated into existing competition workflows with minimal adjustment.

User Requirements

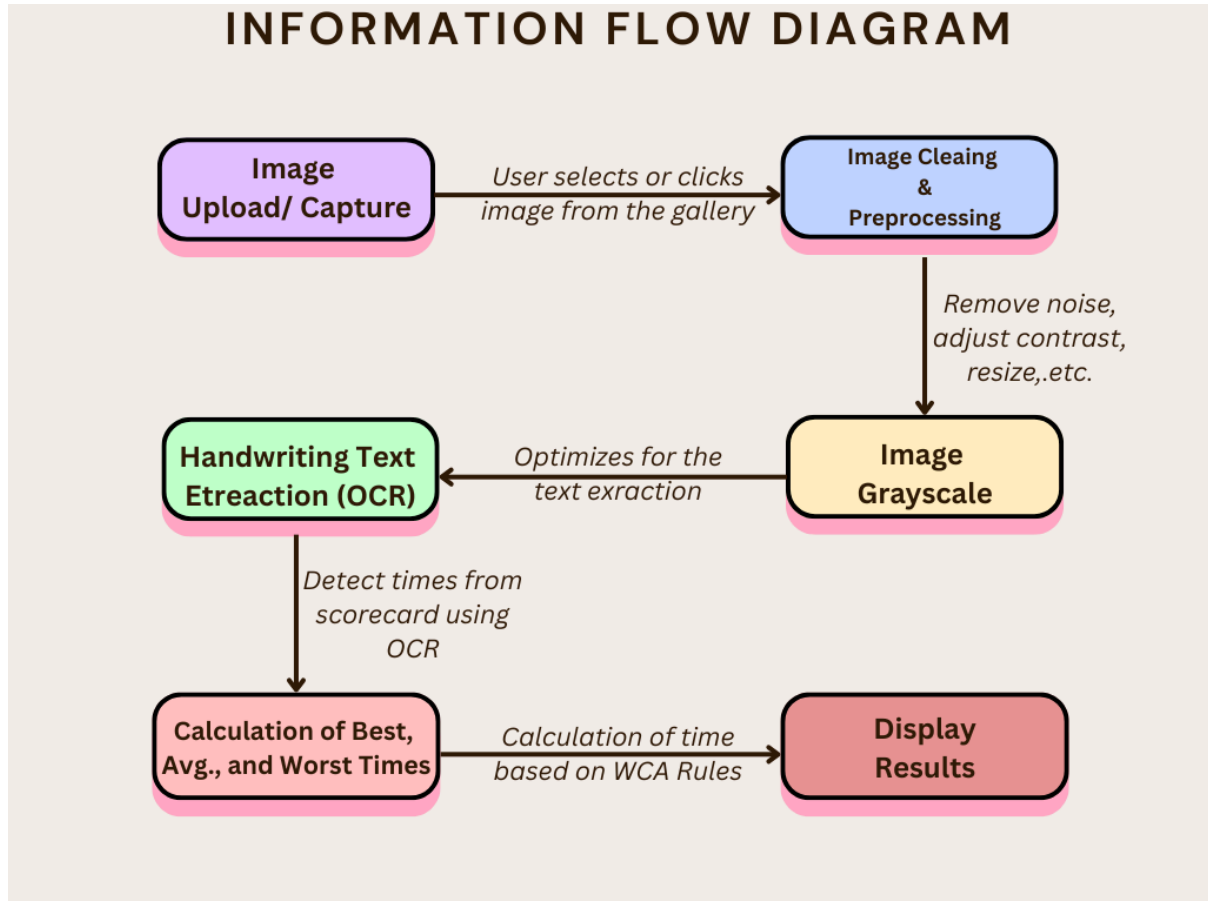
1. Hardware Requirements

- **Smartphone** with:
 - Working **camera** (minimum 5 MP recommended)
 - Sufficient storage to install and run the app
- **Stable Wi-Fi connection** to connect to the same network as the backend server (in the current version)
- **Basic lighting** for capturing clear images of the scorecard

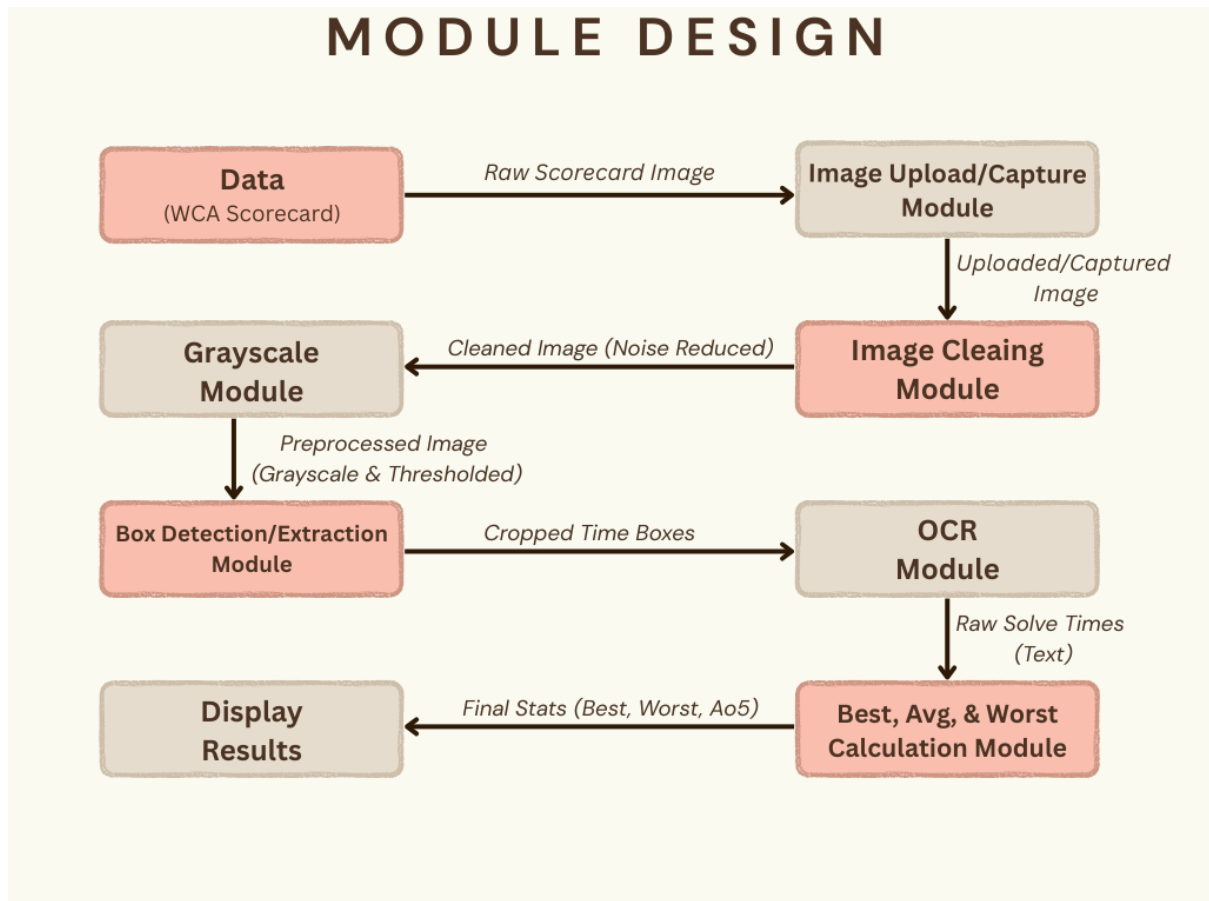
2. Software Requirements

- An **Android device** with support for installing the app
- **Operating system:**
 - Android 8.0 (Oreo) or later
- **Internet access** (to communicate with the backend server)
- **Permissions** granted to:
 - Access the **camera** (for capturing images)
 - Access **storage/gallery** (for uploading images)

System Flow Chart



Module Design



Future Enhancements

- **Cloud-based Backend Hosting**

- Move the Flask backend from a local laptop to a cloud server (e.g., AWS, Heroku, or Firebase Functions) to eliminate the same-network limitation and allow global access.

- **Offline OCR Processing**

- Integrate OCR capabilities directly into the mobile app (e.g., using Tesseract compiled for Android/iOS or ML Kit by Google) to remove the need for a backend in simple use cases.

- **User Profile & History Tracking**

- Add user login and authentication so players can track their past solve times, averages, and progress over time.

- **Event Type Customization**

- Allow users to choose different WCA event formats (2x2, 3x3, 4x4, etc.) with appropriate average calculations (e.g., Ao5, Ao12, Mo3).

- **Real-time Sync with WCA Tools**

- Sync results with official WCA scrambler/timer tools or integrate with WCA's official API (if available) for validation and consistency.

- **Enhanced Image Correction**

- Improve image preprocessing (e.g., auto-cropping, skew correction, noise reduction) for better OCR accuracy on poorly lit or angled photos.

- **Admin Panel for Organizers**

- Build a companion web dashboard for competition organizers to manage submissions, verify scores, and export results.

Conclusion

The development of *CubeAvg* has been a significant step in addressing a niche but meaningful challenge within the speedcubing community, automating the extraction and analysis of WCA scorecards. By integrating image processing techniques and OCR technology with mobile app development, I have created a functional tool that minimizes manual entry, streamlines time calculations, and provides immediate feedback to the user.

Throughout this project, I gained a practical understanding of how to bridge frontend and backend systems using Flutter and Flask, respectively. I explored the real-world complexities of image preprocessing, such as contour detection, adaptive thresholding, and string parsing, to accurately extract handwritten times from photos of physical scorecards. While there were several technical challenges, particularly in handling OCR inconsistencies and ensuring clean data for average time calculation, I addressed these through iterative testing, input validation, and targeted preprocessing improvements.

Although the application currently relies on a locally hosted backend, this architecture lays the foundation for scalable deployment via cloud hosting in the future. It also opens avenues for additional features such as offline OCR, user profiles, and multi-event support. These enhancements could significantly improve usability for both casual cubers and official competition organizers.

This project has not only strengthened my programming and problem-solving skills but also provided me with insights into system design, user experience considerations, and the constraints of mobile-device-based computing.

In conclusion, *CubeAvg* reflects my ability to design, implement, and refine a complete software system from the ground up, and it stands as a valuable contribution to my academic journey and technical development.

Bibliography

Websites

WCA Regulations, [05-11-2024, 7:00 pm]

<https://www.worldcubeassociation.org>

W3 School, [20-11-2024, 6:00pm]

<https://www.W3Schoolsw3schools.com/python/default.asp>

TensorFlow ML Resources, [11-01-2025, 4:00 pm]

<https://www.tensorflow.org>

GeeksforGeeks, [11-03-2025, 1:15pm]

<https://www.geeksforgeeks.org/python-pil-getpixel-method>

GeeksforGeeks, [17-03-2025, 7:30pm]

<http://geeksforgeeks.org/finding-the-size-resolution-of-image-in-python>

GeeksforGeeks, [17-03-2025, 7:34pm]

<https://www.geeksforgeeks.org/python-pil-putpixel-method>

Reddit, [07/04/2025, 12:15 pm]

https://www.reddit.com/r/computervision/comments/15er2y7/2023_review_of_tools_for_handwritten_text/

YouTube Videos

Neural Network Python Project - Handwritten Digit Recognition, [06-02-2025, 8:30 pm]

<https://youtu.be/bte8Er0QhDg?si=W8drZZcJTKb8ET97>

What is Numpy and Why?[07-02-2025]

<https://youtu.be/SqhVpJSHuyI?si=QxBzEnob5bgnbG2n>

Master the basics of Conda environments in Python, [09-02-2025, 11:00 am]

<https://youtu.be/1VVCd0eSkYc?si=jsUsc8wlp5Me7Dal>

How to switch between the multiple Java versions(JDK) in windows 10 | Switch between java 8,11,15,17 [09-02-2025, 4:40pm]

<https://youtu.be/A5FHeR3cE-w?si=Ef8feGb7O4PJ2TI2>

Chain Code and Normalize Chain Code with solved example in Hindi | Image Processing Lectures, [18-02-2025, 8:13 pm]

<https://youtu.be/NPyBHVmlyDY?si=XGSnrPFskU9kCajZ>

10. HSV ColorSpace (Advanced Computer Vision using OpenCV Python),

[11-03-2025, 1:00 pm]

<https://youtu.be/N9OZcP3yNvs?si=uZl6rLNUJwljfg5v>

How to find out the pixel values on the image using python|#08, [11-03-2025, 12:45 pm]

<https://youtu.be/t00B7XVqT5s?si=0nwz9pnWiNxlWdfr>

10.6: Pixel Neighbors - Processing Tutorial, [28-03-2025, 7:12 pm]

https://youtu.be/qB3SA43vKYc?si=NFphK_CSoW7KWj-8

10.1: Intro to Images - Processing Tutorial, [28-03-2025, 7:20 pm]

<https://youtu.be/-f0WEitGmiw?si=6xtWhdUs8HV9g4vJ>

First Steps with Transkribus, Transkribus, [07-04-2025, 12:22 pm]

https://youtu.be/lCdL_kKBmN8?si=15Ht4yPxQldZqBnm

krakenOCR, [07-04-2025, 12:34 pm]

<https://kraken.re/main/index.html>

OpenCV, [07-04-2025, 8:30 pm]

https://docs.opencv.org/3.4/d4/d73/tutorial_py_contours_begin.html#:~:text=Contours%20can%20be%20explained%20simply,better%20accuracy%2C%20use%20binary%20images.

Source Code

App Code

main.dart

```
import 'package:flutter/material.dart';  
  
import 'package:image_picker/image_picker.dart';  
  
import 'dart:io';  
  
import 'splash_screen.dart';
```

```
  
  
import 'package:http/http.dart' as http;  
  
import 'package:http_parser/http_parser.dart';  
  
import 'dart:convert';
```

```
  
  
void main() {  
  
  runApp(const MyApp());  
  
}
```

```
  
  
class MyApp extends StatelessWidget {  
  
  const MyApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(  
  
      debugShowCheckedModeBanner: false,  
  
      home: const SplashScreen(),  
  
      theme: ThemeData.dark().copyWith(  
  
        scaffoldBackgroundColor: Colors.black,
```

```

elevatedButtonTheme: ElevatedButtonThemeData(
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.grey[900],
    foregroundColor: Colors.white,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
),
),
),
);
}
}

```

```

class ImagePickerScreen extends StatefulWidget {
  const ImagePickerScreen({super.key});

  @override
  ImagePickerScreenState createState() => ImagePickerScreenState();
}

```

```

class ImagePickerScreenState extends State<ImagePickerScreen> {
  File? _image;

  final picker = ImagePicker();

  Future<void> _getImage(ImageSource source) async {

```

```

final pickedFile = await picker.pickImage(source: source);

if (pickedFile != null) {

    setState(() {

        _image = File(pickedFile.path);

    });

}

}

Future<void> _processImage(File image) async {

    final uri = Uri.parse('http://192.168.29.13:5000/process-image'); //host IP for Home

    // final uri = Uri.parse(

    //   'http://192.168.100.34:5000/process-image'); //host IP for college

    // final uri =

    //   Uri.parse('http://192.168.1.8:5000/process-image'); //host IP for rent

    //final uri = Uri.parse('http://10.0.2.2:5000/process-image'); //host IP for emulator

    final request = http.MultipartRequest('POST', uri);

    request.files.add(await http.MultipartFile.fromPath(

        'image',

        image.path,

        contentType: MediaType('image', 'png'),

    ));

```

```

final response = await request.send();

if (response.statusCode == 200) {
  final resBody = await response.stream.bytesToString();
  final json = jsonDecode(resBody);

  // Safely handle solves array
  final List<dynamic> solvesList = json['solves'] ?? [];
  final formattedSolves = solvesList.map((s) => "- $s s").join("\n");

  final resultText = ""
$formattedSolves

Best: ${json['best']} s
Worst: ${json['worst']} s
Average: ${json['average']} s
""",
;

  if (!mounted) return;

  showDialog(
    context: context,
    builder: (_) => AlertDialog(
      title: const Text("Result"),
      content: Text(resultText),
      actions: [

```

```

        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text("OK"),
        ),
      ],
    ),
  );
} else {
  debugPrint("Failed to process image. Status: ${response.statusCode}");
  if (!mounted) return;
  showDialog(
    context: context,
    builder: (_) => const AlertDialog(
      title: Text("Error"),
      content: Text("Failed to process image."),
    ),
  );
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('CubeAvg',
        style: TextStyle(fontWeight: FontWeight.bold)),
    ),
  );
}

```

```

backgroundColor: Colors.black,

centerTitle: true,

),

body: Padding(

padding: const EdgeInsets.all(20.0),

child: Column(

mainAxisAlignment: MainAxisAlignment.center,

children: <Widget>[

  _image == null

    ? const Icon(Icons.image, size: 100, color: Colors.grey)

    : AspectRatio(

      aspectRatio: 3 / 4,

      child: ClipRRect(

        borderRadius: BorderRadius.circular(12),

        child: Image.file(

          _image!,

          width: double.infinity,

          fit: BoxFit.cover,

        ),

      ),

    ),

  const SizedBox(height: 20),

  Row(

    mainAxisAlignment: MainAxisAlignment.spaceEvenly,

    children: [

      ElevatedButton.icon(

```

```

        onPressed: () => _getImage(ImageSource.camera),

        icon: const Icon(Icons.camera_alt),

        label: const Text('Capture'),

    ),

    ElevatedButton.icon(

        onPressed: () => _getImage(ImageSource.gallery),

        icon: const Icon(Icons.photo_library),

        label: const Text('Gallery'),

    ),

  ],

),

const SizedBox(height: 20),

ElevatedButton.icon(

    onPressed: _image != null ? () => _processImage(_image!) : null,

    icon: const Icon(Icons.calculate),

    label: const Text('Calculate'),

),

],

),

),

);

}

}

```

splash_screen.dart

```
import 'package:flutter/material.dart';
```

```
import 'dart:async';
```

```
import 'main.dart';
```

```
class SplashScreen extends StatefulWidget {
```

```
  const SplashScreen({super.key});
```

```
  @override
```

```
  // ignore: library_private_types_in_public_api
```

```
  _SplashScreenState createState() => _SplashScreenState();
```

```
}
```

```
class _SplashScreenState extends State<SplashScreen> {
```

```
  @override
```

```
  void initState() {
```

```
    super.initState();
```

```
    Timer(const Duration(seconds: 1), () {
```

```
      Navigator.pushReplacement(
```

```
        context,
```

```
        MaterialPageRoute(builder: (context) => const ImagePickerScreen()),
```

```
      );
```

```
    });
```

```
}
```

```
  @override
```

```

Widget build(BuildContext context) {

  return Scaffold(

    backgroundColor: const Color.fromARGB(255, 0, 0, 0),

    body: Center(

      child: Column(

        mainAxisAlignment: MainAxisAlignment.center,

        children: [

          Image.asset(

            'assets/images/logo.png', // Ensure this image exists

            width: 150,

            height: 150,

          ),

          const SizedBox(height: 20),

          const CircularProgressIndicator(),

        ],

      ),

    ),

  );

}

```

Backend code (Flask)

app.py

```
from flask import Flask, request, jsonify

from PIL import Image

import Text_extract # This is your custom logic file


app = Flask(__name__)


@app.route('/process-image', methods=['POST'])
def process_image():

    if 'image' not in request.files:

        return jsonify({'error': 'No image uploaded'}), 400


    image_file = request.files['image']

    image = Image.open(image_file.stream)

    print("Image received!")


    # Call your processing function from your_processing_script.py

    result = Text_extract.process(image)


    return jsonify(result)


if __name__ == '__main__':

    app.run(host='0.0.0.0', port=5000, debug=True)
```

Text_extract.py

```
import cv2

import numpy as np

import pytesseract

import re

from PIL import Image


def process(pil_image):

    def time_str_to_seconds(t):

        t = str(t).strip().replace(',', '.').replace(" ", "")

        t = re.sub(r'^0-9:.', '', t)

        if t.isdigit():

            if len(t) == 3:

                t = f"{t[0]}.{t[1:]}"

            elif len(t) == 4:

                t = f"{t[:2]}.{t[2:]}"

            elif len(t) == 5:

                t = f"{t[:2]}.{t[2:4]}.{t[4]}0"

            elif len(t) == 6:

                t = f"{t[:2]}.{t[2:4]}.{t[4:]}"

        if ':' in t:

            try:

                mins, secs = t.split(':')

                return float(mins) * 60 + float(secs)
```

```

except:

    raise ValueError(f"Invalid time: {t}")

return float(t)

def format_result_time(t):

    if t >= 60:

        mins = int(t // 60)

        secs = t % 60

        return f"{mins}:{secs:05.2f}"

    else:

        return f"{t:.2f}"

img = cv2.cvtColor(np.array(pil_image), cv2.COLOR_RGB2BGR)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

thresh = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C,

                               cv2.THRESH_BINARY_INV, 15, 10)

contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

rects = []

for cnt in contours:

    x, y, w, h = cv2.boundingRect(cnt)

    area = w * h

    aspect_ratio = w / float(h)

```

```

if 1000 < area < 15000 and 2.0 < aspect_ratio < 6.0:

    rects.append((x, y, w, h))

rects = sorted(rects, key=lambda r: r[1])

skip_indices = [0, 6]

ocr_results = []

for i, (x, y, w, h) in enumerate(rects):

    if i in skip_indices:

        continue

    cropped = img[y:y+h, x:x+w]

    text = pytesseract.image_to_string(cropped)

    cleaned_text = text.strip().replace("\n", "")

    ocr_results.append(cleaned_text)

num_array = []

for x in ocr_results:

    try:

        seconds = time_str_to_seconds(x)

        num_array.append(seconds)

    except ValueError:

        continue

if len(num_array) < 3:

    return {"error": "Need at least 3 valid solve times"}

```

```

sorted_times = sorted(num_array)

best = round(sorted_times[0], 2)

worst = round(sorted_times[-1], 2)


middle_times = sorted_times[1:-1] if len(sorted_times) > 2 else sorted_times

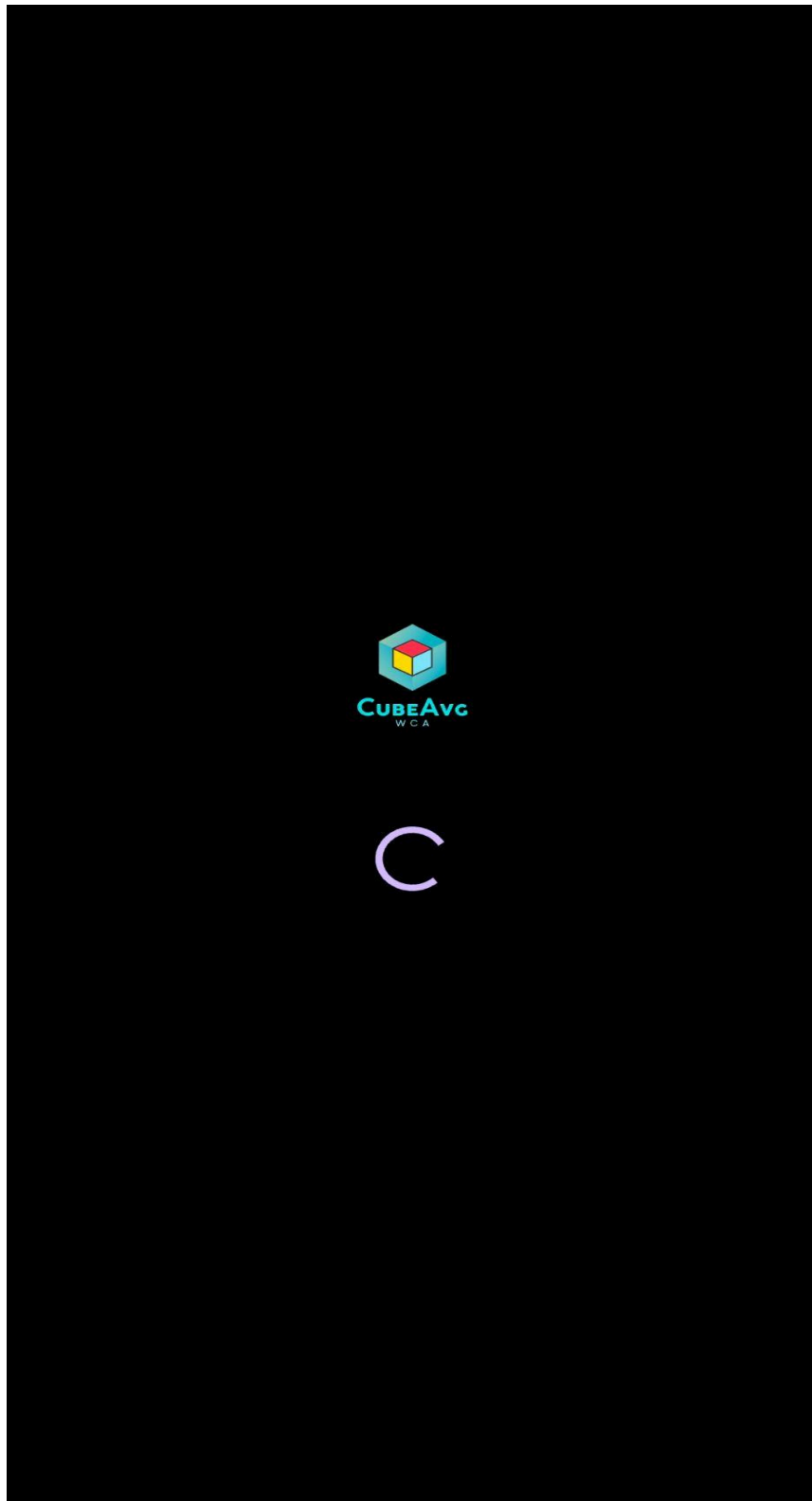
avg = round(sum(middle_times) / len(middle_times), 2)


solves = [format_result_time(t) for t in num_array]


return {
    "solves": solves,
    "best": format_result_time(best),
    "worst": format_result_time(worst),
    "average": format_result_time(avg)
}

```

Sample Input/Output Screens



Splash Screen

CubeAvg

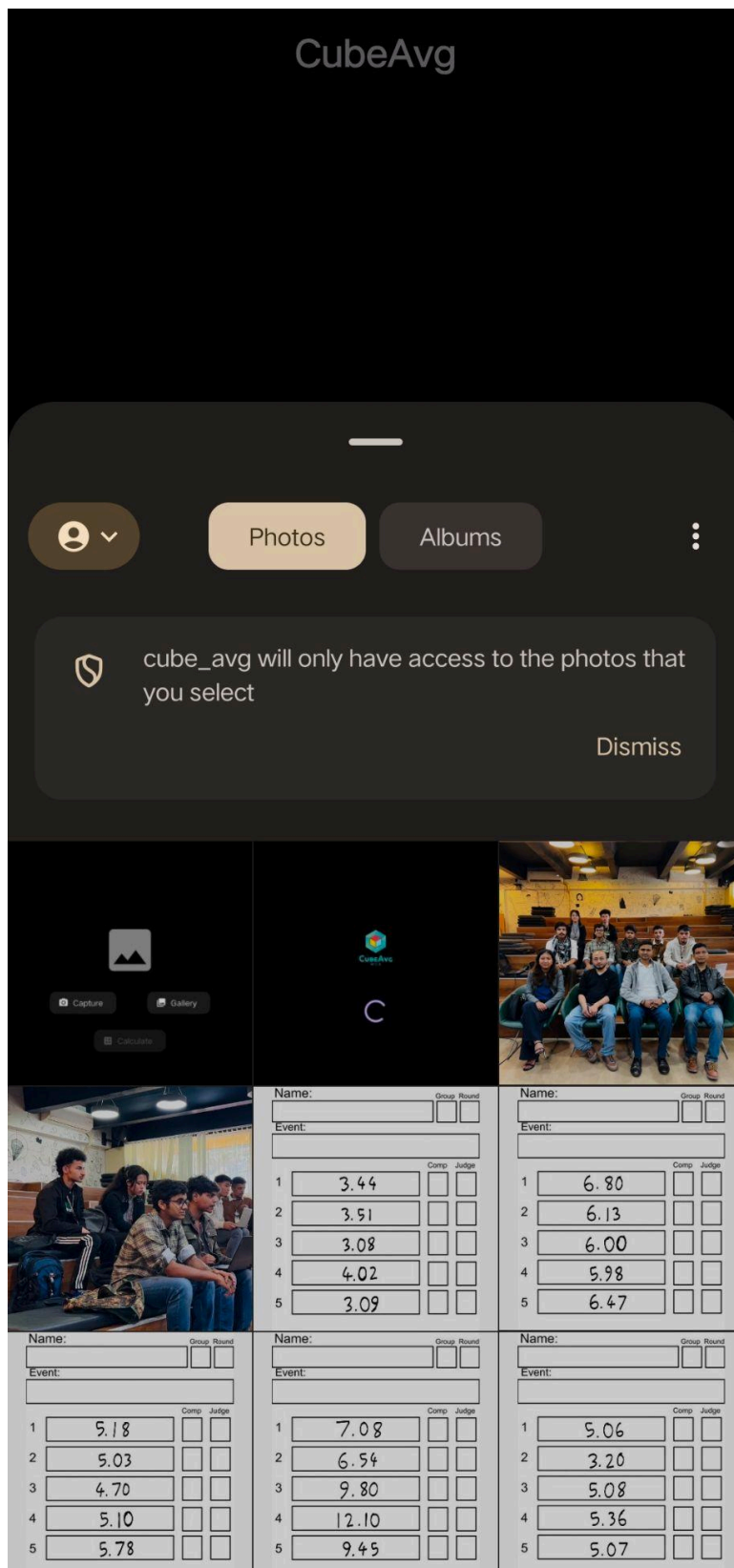


 Capture

 Gallery

 Calculate

Home Screen



Upload from the gallery

CubeAvg

Cubing competition

Name: Group Round

Event:

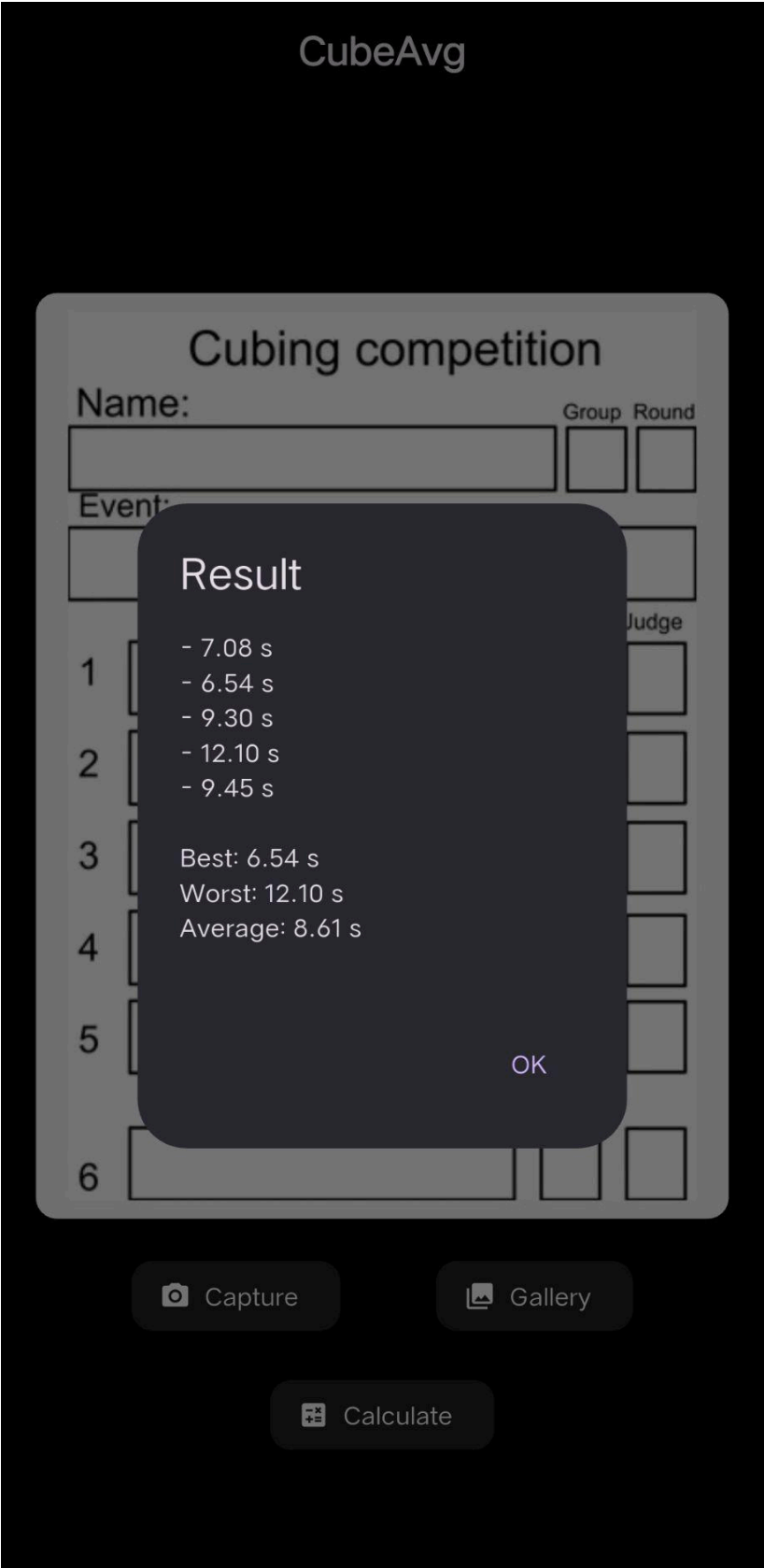
		Comp	Judge
1	7.08		
2	6.54		
3	9.80		
4	12.10		
5	9.45		
6			

 Capture

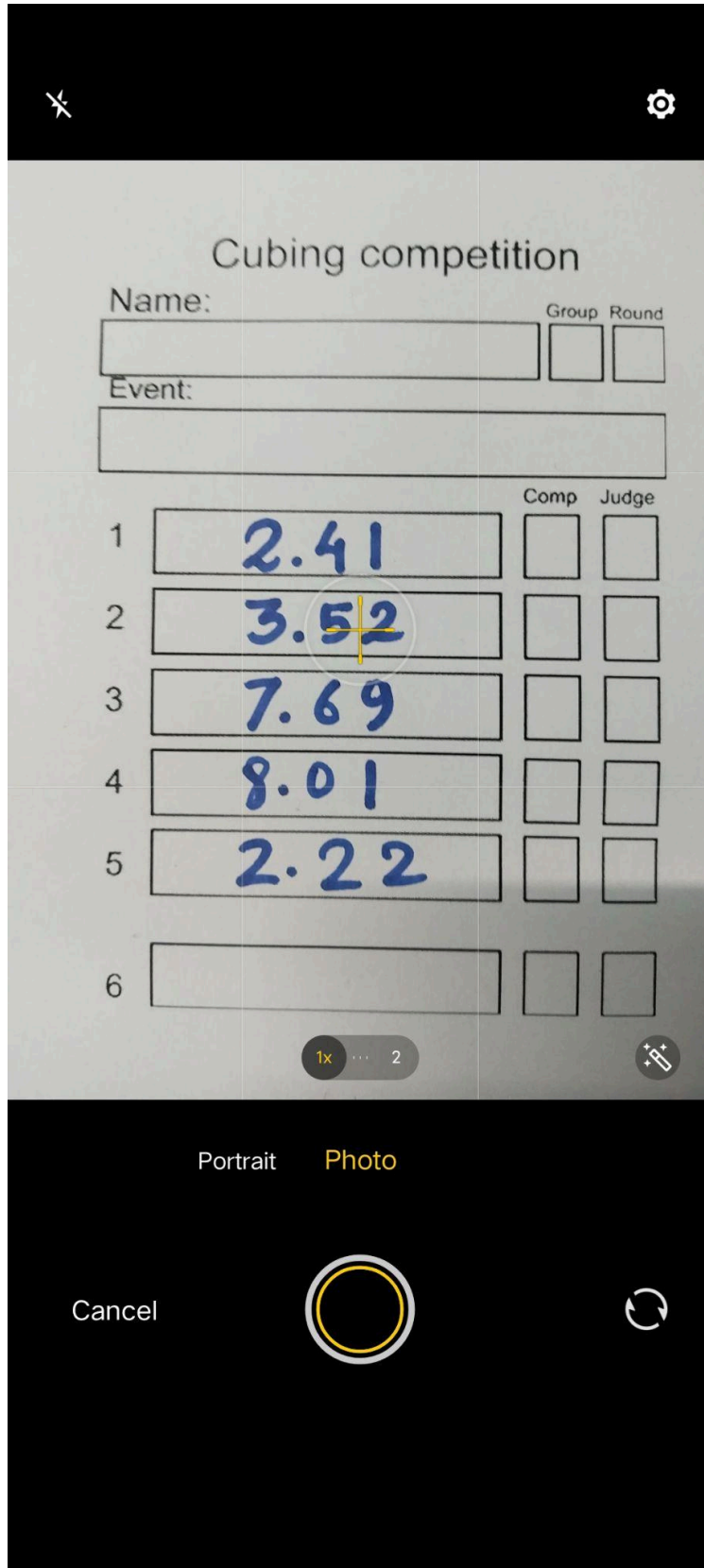
 Gallery

 Calculate

Scorecard Uploaded



Results after Calculation



Capturing the Scorecard directly

Cubing competition

Name:

Group Round

Event:

1

Comp

Judge

2

3

4

5

6

Cancel



OK

Confirmation Page

CubeAvg

Cubing competition

Name: Group Round

Event:

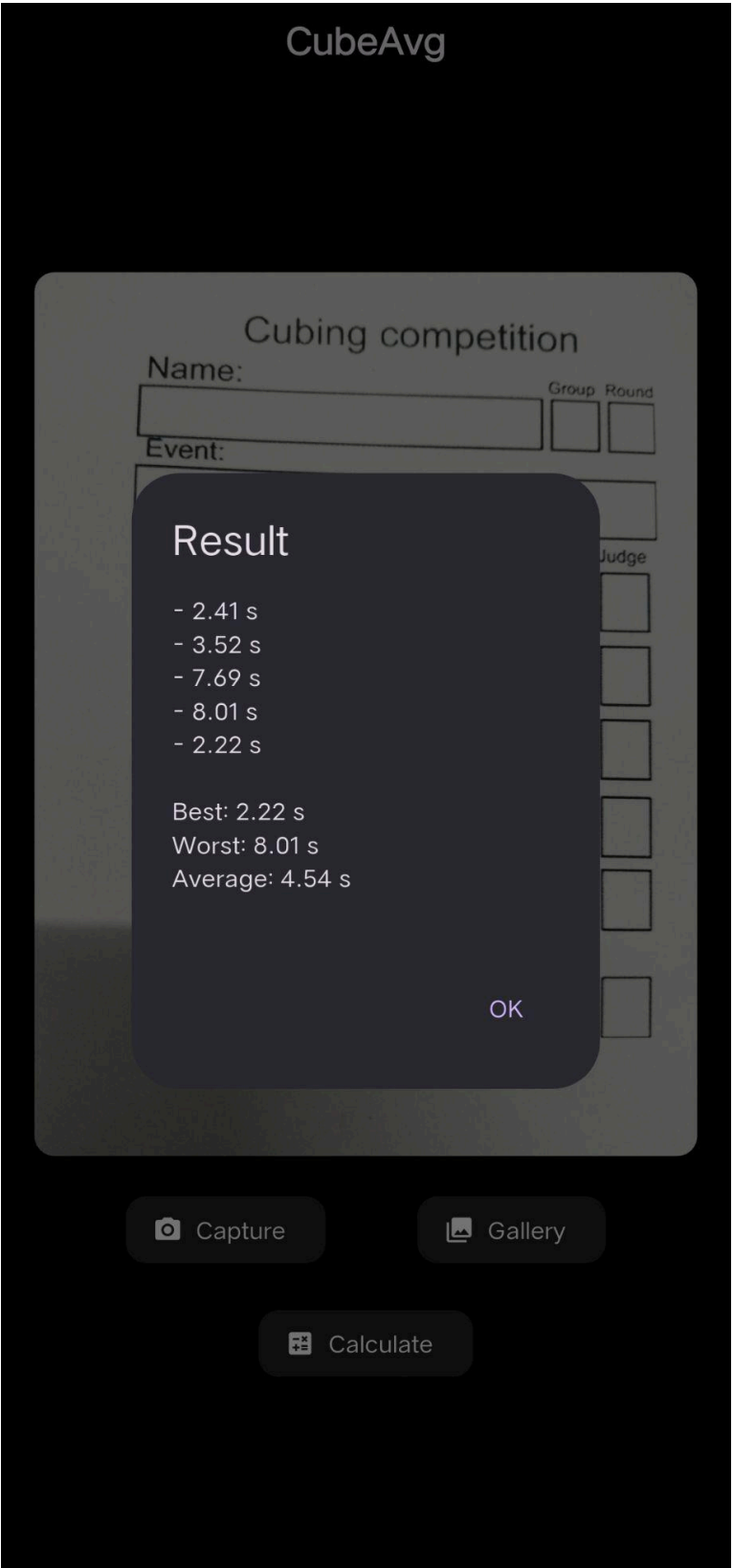
		Comp	Judge
1	2.41		
2	3.52		
3	7.69		
4	8.01		
5	2.22		
6			

 Capture

 Gallery

 Calculate

Scorecard Uploaded



Results after Calculation